

Technical Assignment

Browser-Based C Development Environment for Embedded Targets

Candidate: Tanishq Anoop Khandar

Issued on: 22 July 2026

Due by: 30 July 2026

1. Purpose of this exercise

This assignment exists so we can see how you think and build as we are interested in the reasoning behind your choices than in a feature-complete product.

You will not be penalised for scope you consciously decided to cut. You **will** be asked about it, so keep notes on what you dropped and why.

2. The problem

We want a web application; a browser-based environment where a user can define a workspace, load an ECU description file (.a2l), write C code against the signals it declares, compile for an ARM target, and keep the resulting binary somewhere shareable (preferably a marketplace within the webapp).

Build a working slice of that system.

3. Mandatory constraints

Area	Requirement
------	-------------

Backend	Flask
---------	--------------

Frontend	React
----------	--------------

Compiler target **ARM-based GCC toolchain** ([Link](#) to the opensource compiler)

Deployment	Hosted and reachable by evaluators over the public internet
------------	--

Everything else — database, storage, container runtime, editor component, auth library, CI, hosting provider — is **your** call. Free tiers are entirely acceptable; we are not asking you to

spend money. If cost or platform limits forced a design compromise, say so in your README and it will be treated as a legitimate constraint, not a shortcoming.

4. Functional requirements

4.1 Authentication

Provide a mechanism to identify and separate users, to whatever extent is practical within the time budget.

We are not looking for an enterprise IdP integration. A sound, honestly-scoped implementation with a clear statement of its limitations beats an ambitious one that leaks sessions. Whatever you choose, be explicit in the README about:

- how credentials or tokens are stored and validated
- what a workspace's isolation boundary actually is
- what you would change before this touched real users

4.2 Workspaces

An authenticated user can create and open a named workspace. A workspace is the unit that owns:

- one uploaded reference file
- the source code being edited
- the compilation history and resulting artifacts

Multiple workspaces per user should be possible. How you model this is up to you.

4.3 Reference file upload (.a2l) – Attached as part of the assignment .zip file

The user uploads an **A2L file** — the ASAM MCD-2 MC standard description of an ECU's measurement and calibration interface. Relevant background if this format is new to you:

- It is a structured text format organised into a PROJECT → MODULE hierarchy.
- MEASUREMENT blocks describe readable signals: name, datatype (UBYTE, SWORD, FLOAT32_IEEE, ...), conversion method, limits, and usually an ECU_ADDRESS.
- CHARACTERISTIC blocks describe writable calibration parameters — values, curves, maps.

- COMPU_METHOD blocks define the raw-to-physical conversion (often a linear COEFFS rational function).

You are not expected to write a complete A2L parser. Handling MEASUREMENT and CHARACTERISTIC names, datatypes, and addresses is sufficient. Graceful, informative failure on the parts you don't support is worth more than brittle coverage of the parts you do.

Sample A2L file is shared with you as part of the attachment

4.4 Expose signals as an API

Parsed signals must be available and consumable in the editor as inputs and outputs of the user's C program. At minimum, expose an endpoint returning the signal set for a workspace, with enough metadata (name, direction, datatype, limits) to be useful.

How the signals actually reach the compiled code is a design decision of your choice. Reasonable approaches include generating a header file from the A2L, providing accessor stubs, or injecting declarations at compile time. Pick one, implement it, and justify it.

4.5 Code editor

A usable in-browser C editor. Syntax highlighting is expected; anything beyond that is a bonus, not a requirement. Use an existing editor component — writing one from scratch is not the point of this exercise.

The available signals should be discoverable from within the editing experience.

4.6 Compilation

Trigger a build against the ARM GCC toolchain from the frontend and **stream or display the compiler output in the UI** — including warnings and errors, not just a pass/fail badge.

Compiling code submitted by an authenticated stranger on your server is the sharp edge of this assignment. We would like to see that you noticed. Whatever mitigation you choose — containers, resource limits, timeouts, a restricted filesystem, a queue with a bounded worker pool — document your threat model and what it does and does not cover. "I identified X, mitigated Y, and here is why Z is still open" is a strong answer.

4.7 Artifact marketplace

Successful builds produce a binary that is stored and browsable within the app. Each entry should carry, at minimum:

- the binary itself, downloadable

- the full compilation log for that build
- creation date and time
- build duration
- the workspace and user it originated from

"Marketplace" here means a shared, browsable catalogue of build artifacts. Interpret the visibility and sharing model as you see fit — just tell us what you decided.

5. Explicitly out of scope

To save you time, none of the following are required or rewarded:

- Actual flashing, hardware-in-the-loop, or ECU communication
 - Full ASAM A2L specification coverage
 - Multi-user real-time collaborative editing
 - Payment, licensing, or rating features in the marketplace
 - Polished visual design (clear and functional is enough)
 - Comprehensive test coverage — a few meaningful tests over the parser and API beat a high coverage number
-

6. Deliverables

1. **A hosted, reachable URL** plus credentials for at least two test accounts.
2. **A source repository** (public, or private with access granted to specific users) containing the complete history of your work. Please commit incrementally rather than in one drop — we like to see how a project evolved.
3. **A README** covering:
 - how to run it locally, from a clean checkout
 - an architecture overview, including a diagram if it helps
 - key design decisions and the alternatives you rejected
 - your security posture around compilation, stated honestly

- known limitations, bugs, and what you cut for time
 - roughly how long you spent, and where it went
4. **A short walkthrough** — either a 5–10 minute screen recording or a set of annotated screenshots — showing the flow end to end: log in → create workspace → upload A2L → see signals → write code → compile → view logs → find the artifact in the marketplace.
-

7. Ground rules

- **AI assistants are permitted.** We use them too. The expectation is simply that you understand and can defend every line you submit — we will go through parts of it with you.
 - **Third-party libraries are permitted and encouraged.** Don't rebuild an editor, an auth library, or a parser generator from scratch.
 - **Questions are welcome and cost you nothing.** Ambiguity in this brief is partly deliberate; asking good clarifying questions is a positive signal, not a negative one. Reach out to *[contact]*.
 - **Keep it your own work.** Reused snippets should be attributed in the README.
 - Your submission remains yours. We use it solely for evaluation and will not retain it beyond the hiring decision.
-

Give us a heads-up when you are ready and until then - Good luck. We're looking forward to seeing what you build.